

Disciplina:
Arquiteturas de Softwares

Prof. Esp. Leonardo Dias
Curso de Sistemas de Informação

Padrões de Design

Dias 07 e 08

Disciplina: Arquiteturas de Softwares

Arquitetura não é só “um desenho bonito”, mas sim a estrutura que sustenta o software.

Padrões de Design na Prática

O projeto **Pedido Simples** (<https://github.com/leodias/pedido-simples>) demonstra, de forma simples e prática, **seis padrões de design** usados tanto no **front-end** quanto no **back-end**.

A aplicação simula a criação de pedidos de produtos com desconto, salvando e notificando automaticamente as ações realizadas.

A seguir, são apresentados os arquivos do projeto e o papel de cada padrão de design aplicado:

configuracao.js - Singleton

O que faz:

Garante que exista **somente uma configuração global** compartilhada em toda a aplicação.

Papel do padrão:

Controlar o **acesso único** a um objeto comum — útil para configurações, conexões ou logs.

Exemplo prático:

A classe `Configuracao` guarda a taxa de desconto padrão e é usada em todo o sistema.

Mesmo que seja importada várias vezes, a instância é sempre a mesma.

Quem usa:

`estrategiaDesconto.js` consulta a taxa de desconto aqui.

fabricaProduto.js - Factory

O que faz:

Centraliza a **criação de objetos** de forma padronizada, evitando repetição de código.

Papel do padrão:

Simplificar e **encapsular a lógica de criação** de objetos, deixando o código principal mais limpo.

Exemplo prático:

A função `criarProduto()` cria produtos diferentes conforme o tipo (eletrônico, vestuário etc).

Quem usa:

`app.js` utiliza a fábrica para criar produtos prontos.

estrategiaDesconto.js - Strategy**O que faz:**

Permite **trocar o algoritmo de cálculo de desconto** sem alterar o resto da aplicação.

Papel do padrão:

Isolar **comportamentos intercambiáveis**, permitindo escolher a estratégia em tempo de execução.

Exemplo prático:

`DescontoPadrao` usa o valor configurado no Singleton, enquanto `DescontoFidelidade` aplica um desconto fixo de 10%.

Quem usa:

`app.js` aplica o desconto usando a estratégia desejada.

observador.js - Observer**O que faz:**

Permite que partes do sistema **se inscrevam e sejam notificadas automaticamente** quando algo acontece.

Papel do padrão:

Promover o **desacoplamento entre eventos e reações**, deixando o código mais flexível e extensível.

Exemplo prático:

Quando um pedido é criado, todos os “ouvintes” cadastrados recebem a notificação.

Quem usa:

`comandoCriarPedido.js` envia a notificação.

`app.js` define as ações que devem ocorrer ao receber o aviso.

repositorioPedido.js - Repository**O que faz:**

Gerencia o **armazenamento e recuperação** dos pedidos.

Papel do padrão:

Servir de **camada intermediária entre a aplicação e a fonte de dados**, isolando o acesso ao armazenamento.

Exemplo prático:

O repositório mantém uma lista de pedidos e fornece métodos `salvar()` e `listar()`.

Quem usa:

`comandoCriarPedido.js` salva os pedidos através do repositório.

comandoCriarPedido.js - Command**O que faz:**

Encapsula a ação de **criar um pedido** dentro de um objeto.

Papel do padrão:

Transformar uma operação em um **objeto executável**, facilitando o controle, repetição ou agendamento da ação.

Exemplo prático:

O método `executar()` cria o pedido, salva no repositório e dispara uma notificação.

Quem usa:

`app.js` cria e executa o comando.

Explicação detalhada dos padrões de design aplicados ao projeto

Veremos agora cada padrão aplicado aos arquivos do projeto **pedido-simples/**.

Para cada arquivo, é descrito **qual problema o padrão resolve, como ele foi implementado no código, quem o usa no fluxo da aplicação, vantagens e limitações / sugestões de melhoria.**

configuracao.js - Singleton

Problema que resolve

Centralizar configurações compartilhadas (ex.: moeda, percentual de desconto padrão) e garantir que toda a aplicação use a mesma instância/estado.

Como está implementado

No construtor a classe verifica if (`Configuracao.instancia`) return `Configuracao.instancia`; e depois atribui `Configuracao.instancia = this`. O módulo exporta `module.exports = new Configuracao()`; — assim qualquer `require("./configuracao")` recebe sempre a mesma instância.

Quem usa / fluxo

Outros módulos que precisam de configuração (por exemplo `estrategiaDesconto.js`) importam essa instância e leem `config.percentualDescontoPadrao`.

Vantagens

- Estado global centralizado e consistente.
- Fácil acesso sem passar objeto de config por parâmetro por toda a aplicação.

- Simples de implementar em Node/JS.

Limitações / melhorias

- Pode tornar testes mais difíceis (estado global compartilhado).
Solução: oferecer método para resetar o estado em testes.
- Em apps maiores, considerar injeção de dependência em vez de singleton puro para maior testabilidade.
- Evitar armazenar lógica complexa/state mutável sem controle (adicionar getters/setters e validação se necessário).

fabricaProduto.js - Factory (simples)

Problema que resolve

Esconder a lógica de criação de objetos quando há variações (produto eletrônico, vestuário, etc.), evitando condicionais espalhadas pelo código.

Como está implementado

Função criarProduto(tipo, nome, preco) retorna objetos com atributos diferentes conforme tipo. Exporta { criarProduto }.

Quem usa / fluxo

app.js chama criarProduto(...) para obter instâncias de produto já montadas (com garantiaMeses ou tamanho etc).

Vantagens

- Centraliza regras de construção de produto.
- Facilita adicionar novos tipos sem alterar consumidores.
- Mantém app.js limpo (não precisa conhecer detalhes de construção).

Limitações / melhorias

- Implementação atual retorna objetos literais; se precisar de comportamento (métodos), poderia retornar instâncias de classes (ex.: class ProdutoEletronico extends Produto).

- Para maior escalabilidade, transformar em fábrica abstrata com registro de “criadores” (mapa tipo→função) para evitar if/else.

estrategiaDesconto.js - Strategy

Problema que resolve

Permitir trocar a forma de calcular o desconto sem modificar quem consome o resultado (separação de algoritmos).

Como está implementado

Há uma classe base EstrategiaDesconto com método calcular(preco). Duas implementações concretas:

- DescontoPadrao (usa config.percentualDescontoPadrao)
- DescontoFidelidade (desconto fixo de 10%)

Quem usa / fluxo

app.js instancia a estratégia desejada (new DescontoPadrao(config) ou new DescontoFidelidade()) e chama calcular(preco) para obter precoFinal.

Vantagens

- Facilita adicionar novas regras de desconto sem tocar código existente.
- Testes isolados das estratégias são fáceis.
- Permite selecionar estratégia em runtime (ex.: promoção, fidelidade, cupom).

Limitações / melhorias

- Estratégias atuais são pequenas; ideal extrair interface/contrato em TypeScript para segurança.
- Para sistemas com muitas regras, considerar combinar Strategy com Chain of Responsibility (encadear regras) ou Policy Engine.

observador.js - Observer (Publish/Subscribe simples)

Problema que resolve

Desacoplar emissores de eventos dos consumidores: permitir que várias partes do sistema reajam à ocorrência de um evento sem conhecerem umas às outras.

Como está implementado

Classe Observador que guarda inscritos (array de funções). Métodos inscrever(funcao) e notificar(dado) que chama cada função inscrita.

Quem usa / fluxo

comandoCriarPedido.js chama observador.notificar(...) quando um pedido é criado; app.js faz observador.inscrever(...) para registrar log/notify functions.

Vantagens

- Baixo acoplamento entre produtores e consumidores.
- Fácil adicionar novos "ouvintes" (email, log, analytics) sem modificar o produtor.
- Simples de entender e usar.

Limitações / melhorias

- Implementação básica: não tem tópicos/nomes de evento (todos os inscritos recebem tudo). Para maior flexibilidade, implementar on(evento, handler) e emit(evento, dado).
- Sem tratamento de erros: uma exceção em um listener interrompe os outros. Recomenda-se executar cada listener em try/catch.
- Para grandes sistemas, considerar uso de um pub/sub mais robusto ou EventEmitter do Node (suporta nomes de eventos e listeners).

repositorioPedido.js - Repository

Problema que resolve

Abstrair o acesso aos dados (persistência): a aplicação usa métodos do repositório (salvar, listar) sem se importar se os dados estão em memória, arquivo ou banco de dados.

Como está implementado

Classe RepositorioPedido com `this.pedidos = []`, método `salvar(pedido)` que empurra no array e `listar()` que retorna o array. Exporta `new RepositorioPedido()` (instância única).

Quem usa / fluxo

`comandoCriarPedido.js` chama `repositorio.salvar(pedido)` para persistir; `app.js` chama `Repositorio.listar()` para exibir pedidos.

Vantagens

- Isola lógica de persistência; facilita trocar a implementação (por exemplo, migrar para MongoDB) sem mudar a lógica de negócio.
- Facilita testes (mockar o repositório).
- Responsabilidade única: só lida com armazenamento.

Limitações / melhorias

- Implementação em memória — não é persistente entre execuções.
Para produção, implementar adaptadores (ex.: `RepositorioPedidoMongo`, `RepositorioPedidoArquivo`) que implementem a mesma interface.
- Adicionar operações importantes: `buscarPorId`, atualizar, remover, paginação, filtros.
- Tratar concorrência/atomicidade se migrar para banco real.

comandoCriarPedido.js - Command

Problema que resolve

Encapsular uma ação (criar pedido) como um objeto que pode ser executado, enfileirado, logado ou desfeito.

Como está implementado

Classe ComandoCriarPedido recebe pedido no construtor e tem método executar() que chama repositorio.salvar(this.pedido) e observador.notificar(...).

Quem usa / fluxo

app.js cria instâncias do comando (new ComandoCriarPedido(pedido1)) e chama executar() — mas poderia também guardar o comando para histórico, re-executar ou enviar para fila.

Vantagens

- Organiza ações como objetos; facilita executar a mesma ação em diferentes contextos (sync, async, fila).
- Permite padrão “undo” se for implementado método desfazer() (command reversível).
- Integra bem com mecanismos de fila/trabalho (workers) e gravação de histórico (audit).

Limitações / melhorias

- Implementação básica: apenas executar(). Para features avançadas, adicionar validar(), desfazer() e retornar status/resultado.
- Para operações assíncronas (ex.: salvar em DB remoto), executar() deveria ser async e tratar erros/exceções.
- Em arquiteturas mais complexas, combinar Command com Mediator ou Message Queue (RabbitMQ, Bull) para processamento fora do processo principal.

Resumo e relacionamento entre padrões

- **Singleton (configuracao)** fornece parâmetros que **Strategy** usa.
- **Factory (fabricaProduto)** cria objetos que compõem os pedidos.
- **Strategy (estrategiaDesconto)** calcula precoFinal sem o cliente precisar conhecer a fórmula.
- **Command (comandoCriarPedido)** encapsula a operação de criação do pedido e usa **Repository** para persistir e **Observer** para notificar interessados.
- **Repository (repositorioPedido)** abstrai armazenamento.
- **Observer (observador)** desacopla notificações (log, UI, e-mail, etc).

app.js - Aplicação principal

O que faz:

Orquestra todos os padrões em funcionamento.

Fluxo do programa:

1. Cria produtos com o **Factory**.
2. Aplica descontos com o **Strategy**.
3. Monta o pedido.
4. Executa o **Command** para criar o pedido.
5. O **Command** salva o pedido no **Repository** e aciona o **Observer**.
6. O **Observer** informa aos ouvintes (como logs) que um pedido foi criado.